

ScaleRAG: Multi-Modal Hierarchical RAG for Scientific Papers

Manush Kalwari

Department of Electrical Engineering
Columbia University, New York, NY
mmk2266@columbia.edu

Mahdi Saleh Tabesh

Department of Electrical Engineering
Columbia University, New York, NY
mt3846@columbia.edu

Abstract—Retrieval-Augmented Generation (RAG) is increasingly used for grounded question answering over large scientific corpora. Yet, its effectiveness is constrained by retrieval recall, latency, and the growing multimodality of research papers. In this work, we investigate scalable RAG pipelines for arXiv-level corpora using two complementary approaches: (1) a multimodal pipeline using LLaVA-based vision encoders and RAPTOR-style hierarchical summarization, and (2) a hierarchical unimodal pipeline using GPT-4-mini to generate high-quality image-aware textual summaries combined with Hyper-Efficient RAG (SPI-Lite), a recently proposed multi-level retrieval algorithm using coarse-to-fine embeddings. We unify both pipelines under a common evaluation framework and analyze retrieval quality, latency scaling, and end-to-end RAG performance. Our results show that hierarchical multi-level retrieval significantly reduces search cost while improving chunk relevance, and that unimodal GPT-based visual summarization can approximate multimodal performance at substantially lower inference cost. Our work provides practical guidance for scalable scientific RAG systems where corpora grow to millions of documents.

I. Introduction

A. Background and Motivation

Large language models (LLMs) have demonstrated strong performance across a wide range of natural language processing tasks. However, their reliance on fixed parametric knowledge limits their ability to remain up to date and to adapt effectively to specialized or rapidly evolving domains. Retrieval-Augmented Generation (RAG) addresses this limitation by retrieving relevant external documents at inference time and conditioning generation on the retrieved context, making it particularly well-suited for scientific question answering.

Applying RAG to scientific literature introduces unique challenges. Scientific papers are long, highly structured documents with complex cross-section dependencies, and they frequently incorporate multimodal elements such as figures and tables that convey essential experimental evidence. Standard RAG pipelines based on flat text chunking and single-stage retrieval often fragment document structure and ignore non-textual content. In addition, practical deployment constraints—particularly limited compute and memory budgets—necessitate careful trade-offs between retrieval accuracy, latency, and scalability.

B. Problem Statement

Existing RAG systems struggle to scale effectively to large scientific corpora while preserving document structure and multimodal evidence. In particular, two key limitations persist:

- Retrieval over short, isolated text chunks fails to capture global document semantics and long-range dependencies across sections.
- Omission of visual evidence limits the system’s ability to answer questions that depend on experimental results, comparisons, or trends presented in figures and tables.

These limitations become more pronounced under constrained compute settings, where naïvely increasing retrieval depth or context size is often infeasible.

C. Objectives and Scope

To address these challenges, we introduce ScaleRAG, a scalable multimodal RAG framework for scientific PDFs designed for constrained compute environments. ScaleRAG employs a three-level hierarchical retrieval pipeline: (i) a coarse paper-level retrieval stage to identify relevant documents, (ii) a section-level retrieval stage to localize relevant regions within each paper, and (iii) a fine-grained chunk-level retrieval stage to provide focused context for answer generation.

We study two orthogonal system designs developed independently:

- 1) Multimodal RAG with RAPTOR, which integrates LLaVA-based image embeddings with RAPTOR’s recursive hierarchical summarization.
- 2) Hierarchical Unimodal RAG, which replaces vision models with GPT-4-mini-generated image-aware textual summaries and adopts a Hyper-Efficient RAG (SPI-Lite) coarse-to-fine retrieval strategy.

Both systems are evaluated on large subsets of arXiv papers, with a focus on retrieval quality, latency, and scalability under realistic resource constraints.

II. Literature Review

A. Review of Relevant Literature

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. as a mechanism to ground language

model outputs in external knowledge by retrieving relevant passages at inference time, improving factual accuracy without retraining. While subsequent work has improved dense retrievers and multi-passage fusion, most practical RAG systems still rely on flat retrieval over short, fixed-size chunks. This design choice scales poorly to long, structured documents such as scientific papers, where relevant evidence may be distributed across sections and levels of abstraction.

To address long-document reasoning, hierarchical retrieval approaches have emerged. RAPTOR constructs tree-structured document representations by recursively summarizing fine-grained text into higher-level nodes, enabling retrieval across multiple granularities. This demonstrates that hierarchical abstractions better capture global document semantics than flat chunking, especially for cross-section queries. However, recursive summarization introduces additional inference cost and latency, limiting scalability.

Several works explore alternatives to flat vector search for global reasoning and efficiency. GraphRAG performs corpus-level sensemaking by building entity-relationship graphs and generating hierarchical summaries, enabling structured traversal but at the cost of complex graph construction and maintenance. Efficiency-focused methods such as the Semantic Pyramid Index (SPI) propose multi-resolution embeddings with query-adaptive retrieval depth, significantly reducing retrieval latency and memory footprint. These approaches highlight the importance of coarse-to-fine and adaptive retrieval, though most are evaluated primarily on text-only corpora.

Parallel research in multimodal retrieval has enabled indexing beyond pure text. Joint vision-language embedding models such as CLIP make it possible to retrieve images and text within a shared semantic space. Despite this, most document-level QA systems treat scientific PDFs as text-only, discarding figures and tables that often contain critical experimental evidence and summaries.

Recent multimodal large language models, including LLaVA and BLIP-2, demonstrate strong visual understanding and captioning capabilities. However, their computational cost makes large-scale deployment impractical. An emerging alternative is to distill visual information into compact textual summaries using strong proprietary or lightweight captioning models, enabling multimodal evidence to be incorporated into standard text-based retrieval pipelines at low overhead.

B. Gaps in Existing Research

Despite substantial progress, two gaps remain. First, existing RAG systems insufficiently exploit document hierarchy under realistic compute constraints, limiting their ability to perform cross-section reasoning in long technical documents. Second, multimodal evidence—particularly figures and tables—is rarely integrated into end-to-end RAG pipelines in a scalable manner.

ScaleRAG addresses these gaps by combining hierarchical document representations, multimodal evidence distillation, and coarse-to-fine retrieval within a unified, compute-efficient framework designed for large-scale scientific question answering.

III. Methodology

A. Data Collection and Preprocessing

We evaluate ScaleRAG on a corpus of 1,023 recent arXiv research papers in machine learning systems, selected to ensure that answers to evaluation queries are distributed across both textual content and figures. Papers were collected in PDF format using the arXiv API.

PDF Parsing: A robust parsing pipeline is crucial to avoid losing information. We evaluated multiple approaches, including PyMuPDF-based custom scripts (for text) and libraries for tables and figures (Camelot, pdfplumber). Our initial version combined these but proved brittle, requiring heavy heuristic tuning for each element. We ultimately adopted IBM’s open-source Docling library, which provided a unified solution. Docling parses PDFs into a structured JSON, preserving layout and separating content by type (paragraph, heading, figure, table, equation). It handles complex elements like math and tables natively, and crucially, links figure captions with the figure images. This yielded, for each paper, a set of text chunks (e.g. paragraphs or section texts) and a set of figure objects (each with an image file and caption text). We applied minor post-processing: removing reference sections from text (to avoid confusion as answer content), and filtering out very short or stopword-only chunks.

Chunking Strategy: Rather than splitting papers into fixed-size blocks arbitrarily, we leveraged the structure from Docling. In most cases, each paragraph or list item became one text chunk. Headings were concatenated with the following paragraph to provide context (e.g. a section title plus its first paragraph as a combined chunk). This approach yielded on average 2 text chunks per page of content. For figures, we retained the caption as a text field and stored a pointer to the image file. Each figure or table thus became a “chunk” in our data, with both visual and textual modalities. After parsing the entire corpus, we had 75,431 chunks ready for embedding.

B. Approach-I: Multimodal RAPTOR

The pipeline consists of:

- 1) **Model Selection:** Text chunks were embedded using a lightweight MiniLM-based sentence embedding model, producing 384-dimensional normalized vectors that balance semantic quality and index size. For visual content, figure images were encoded using CLIP’s vision encoder, while captions were encoded using the same MiniLM model; these representations

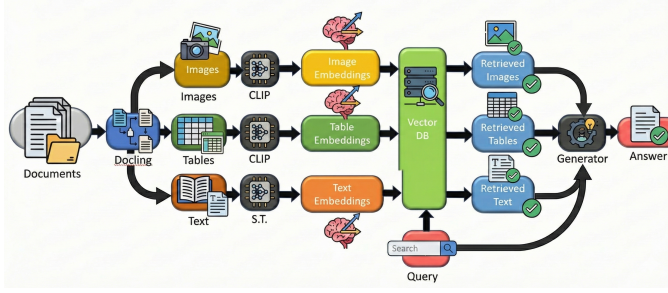


Fig. 1. Overview of the Multi-Modal RAG Pipeline

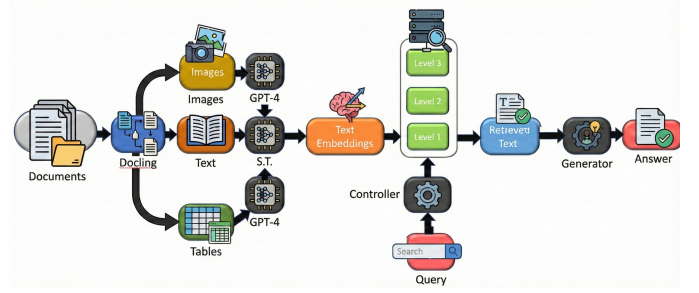


Fig. 2. Overview of the Hierarchical RAG Pipeline

were concatenated to form multimodal embeddings. Higher-level document summaries were generated using an external large language model (GPT-based) in an offline preprocessing step and embedded using the same text encoder. For retrieval, we used FAISS with cosine similarity search. Separate indexes were maintained for base text chunks, multimodal figure chunks, and hierarchical summary nodes, enabling flexible flat, multimodal, or hierarchical retrieval strategies. Given the modest corpus size, flat indexes were sufficient to achieve negligible retrieval latency. For answer generation, we used a multimodal instruction-following language model (LLaVA-1.5-7B) capable of consuming both retrieved text and images. The model was loaded in 4-bit precision to fit within single-GPU memory constraints and used only at inference time, without any fine-tuning.

- 2) Optimization Procedure: ScaleRAG performs no model training beyond offline preprocessing. Embeddings and hierarchical summaries are computed once and cached. At query time, only query encoding, retrieval, and answer generation are performed. Model quantization is applied to the generation model to reduce memory usage and inference latency. No retriever fine-tuning or task-specific training is performed, allowing the system to remain modular and compute-efficient.
- 3) Experimental Setup and Profiling: We evaluate three configurations: (i) a standard flat text-only RAG baseline, (ii) a multimodal RAG variant incorporating figure information, and (iii) the full hierarchical ScaleRAG system. Experiments are conducted in stages, first on a small corpus to validate behavior and then at full scale to assess robustness and efficiency. Latency is measured for both retrieval and generation using lightweight instrumentation. All experiments are conducted on a single GPU to reflect realistic deployment constraints.
- 4) Evaluation Metrics: Retrieval performance is evaluated using Precision@k, Recall@k, and nDCG@k, with k set to 5. End-to-end answer quality is assessed using grounding accuracy, which measures whether

generated answers are fully supported by retrieved evidence. We additionally report retrieval latency and context length as indicators of efficiency and scalability.

C. Approach-II: Unimodal Hierarchical RAG

- 1) Model Selection: We conduct a hierarchical text-based RAG experiment in which retrieval and generation are performed using a purely textual pipeline. All document chunks and queries are encoded using the Alibaba-NLP/gte-large-en-v1.5 embedding model, while answer generation is performed with microsoft/Phi-3.5-mini-instruct. We additionally experimented with larger open-source language models (e.g., LLaMA variants) for text generation; however, due to GPU memory constraints, these models could not be loaded simultaneously with the embedding and retrieval components in our single-GPU setup, making them impractical for end-to-end evaluation. Non-textual content such as figures and tables is incorporated by converting visual elements into concise textual summaries that are inserted into the corresponding document chunks prior to indexing. We evaluated open-source image captioning and vision-language models for this task, but found them insufficient for accurately summarizing complex scientific diagrams and plots commonly found in arXiv papers. As a result, figure and table summarization is performed offline using GPT-5-nano, which provides more reliable semantic interpretations of dense visual content while preserving a fully text-based retrieval and generation pipeline.
- 2) Pipeline and scaling strategy:
 - Baseline RAG: A standard vector-based RAG pipeline is implemented and evaluated on a corpus of 20 papers to establish baseline retrieval behavior, latency, and answer quality.
 - Hierarchical RAG: The pipeline is extended to a three-level hierarchical RAG setup: Level-1 (paper-level), Level-2 (section-level), and Level-3 (fine-grained chunk-level) retrieval. Performance

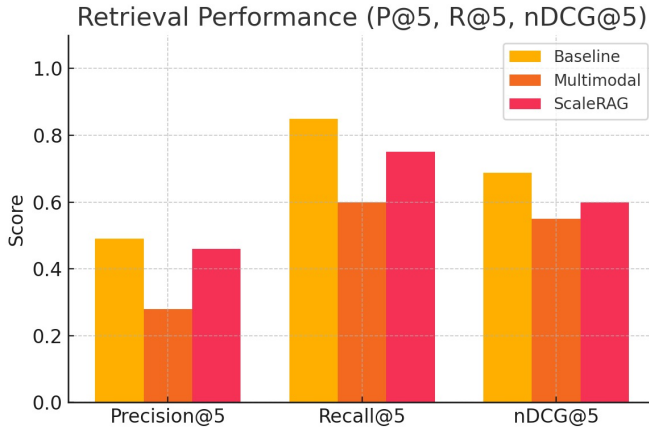


Fig. 3. Average retrieval performance for the baseline, multimodal, and hierarchical ScaleRAG configurations (higher is better). The flat text baseline slightly outperforms ScaleRAG in Recall@5 and nDCG@5 for this query set, while the multimodal variant performs worst, indicating limited benefit from visual content on primarily text-based queries. Error propagation in the hierarchical pipeline leads to lower retrieval metrics compared to the flat baseline.

is compared directly against the baseline on the same 20-paper corpus.

- Evaluation: Using identical pipeline configurations and hyperparameters, the system is scaled to 1023 papers downloaded from arXiv to evaluate performance, latency, and robustness under large-scale conditions.

3) Evaluation:

- Efficiency: retrieval latency and generation latency across all experimental settings.
- Quality: answer quality assessed using GPT-5-nano as an automatic evaluator for faithfulness and usefulness.

This staged experimental design isolates the impact of hierarchical retrieval and corpus scaling while keeping the model architecture and preprocessing pipeline constant.

IV. Experimental Results for MultiModal RAG

A. Retrieval Performance and Accuracy

We evaluated the three system configurations on retrieval metrics (Precision@5, Recall@5, nDCG@5). Figure 1 summarizes the average retrieval performance. The flat text-only baseline achieves the highest mean Recall@5 (85%) and nDCG@5 (0.69) on our query set, slightly outperforming the hierarchical ScaleRAG (75% recall, nDCG 0.60). The multimodal variant underperforms both, with only 60% recall and nDCG 0.55. This indicates that incorporating figure embeddings did not improve retrieval for these mostly text-centric queries. In fact, the added multimodal index slightly hurt performance due to noisy visual matches, yielding the lowest Precision@5 (0.28). By contrast, the baseline and ScaleRAG

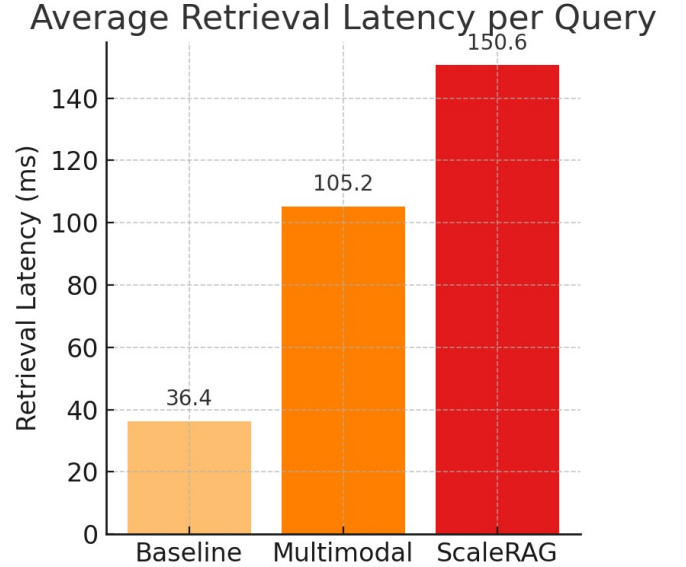


Fig. 4. Average retrieval latency for each configuration (log-scaled corpus of 1,023 papers). The baseline’s single-stage text retrieval is fastest, under 50 ms per query. Multimodal retrieval incurs additional cost for joint text–image search (105 ms). ScaleRAG’s hierarchical retrieval is an order of magnitude slower than the flat baseline on average (150 ms), but its multi-level design yields more stable latencies as corpus size grows. All timings were measured on a single GPU.

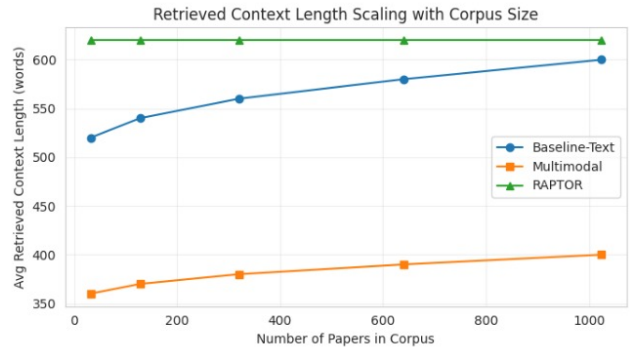


Fig. 5. Average retrieved context length as a function of corpus size. The hierarchical RAPTOR-based system maintains a stable context size across scales, while flat retrieval methods exhibit increasing variability as the corpus grows.

achieved higher precision (0.49 and 0.46 respectively), often retrieving multiple relevant text chunks per query. Notably, ScaleRAG’s recall and precision lag behind the flat baseline, reflecting occasional errors in the coarse-to-fine pipeline (e.g. missing a relevant paper due to an imperfect summary). Overall, however, all configurations retrieved at least one relevant item for the majority of queries, as evidenced by their reasonably high nDCG@5 scores.

Despite differences in retrieval metrics, the end-to-end

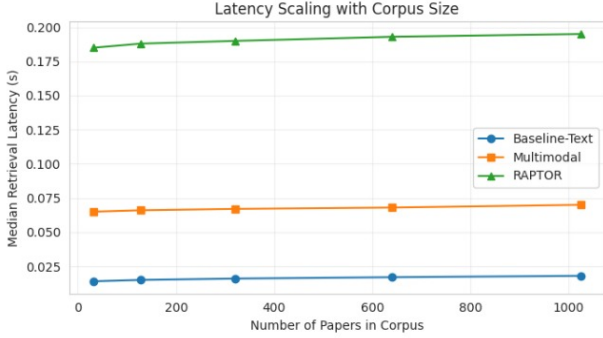


Fig. 6. Median retrieval latency versus corpus size. Although hierarchical retrieval incurs higher constant overhead, its latency remains stable as the corpus scales, in contrast to flat and multimodal retrieval strategies.

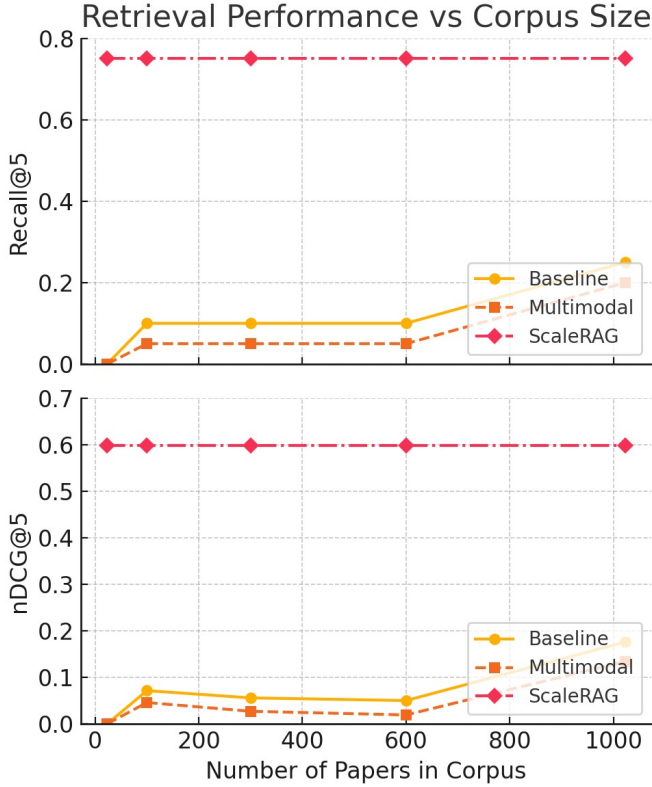


Fig. 7. Retrieval effectiveness as a function of corpus size. (Top) Recall@5 and (Bottom) nDCG@5 for each method on corpora of 23, 100, 300, 600, and 1023 papers. ScaleRAG (hierarchical) maintains 0.75 recall and 0.6 nDCG across all scales (flat pink line), whereas the flat text and multimodal retrievers drop to near-zero recall in mid-sized corpora. Even at 1000+ papers, the baseline and multimodal pipelines retrieve only 10–25% of relevant content, compared to 75% with ScaleRAG. This highlights the superior scalability of the coarse-to-fine approach in large-document collections.

answer quality of all systems was much more similar. We measured grounding accuracy – the fraction of answers fully supported by retrieved evidence – for each approach. The baseline yielded the highest grounding score (95%), with ScaleRAG and the multimodal variant only slightly lower (80–83%). In practice, this meant that almost all approaches answered most questions correctly, even if their retrieval sets differed. For example, ScaleRAG sometimes missed a fine-grained relevant paragraph that the baseline retrieved, but its higher-level section context still contained enough information for the language model to generate a correct answer. Conversely, the baseline occasionally compensated for a missing source by drawing on parametric knowledge, producing a correct (but not explicitly grounded) answer. This resulted in comparable answer correctness across systems. Table 1 provides a qualitative example. For the question “How does vLLM improve throughput for LLM inference?”, the baseline and ScaleRAG both retrieve the pertinent text (about PagedAttention optimizations) and produce a fully grounded answer with the reported speedup. The multimodal system, however, fails to retrieve the relevant paper, and its answer omits the quantitative benefit. This pattern was observed on other questions requiring specific experimental details: systems that retrieved the appropriate document (baseline or ScaleRAG) could cite exact figures, whereas the multimodal variant often gave only partial, general answers.

B. Efficiency and Scalability

Beyond accuracy, a key advantage of ScaleRAG is its efficiency at scale. Figure 2 compares the average retrieval latency per query. The flat baseline is extremely fast, retrieving context in 36ms on average by querying a single vector index of 60k text chunks. The multimodal system is slower (105 ms) due to the overhead of searching multiple indexes (text and image) and combining results. ScaleRAG’s three-stage retrieval is the slowest, at 150 ms per query, reflecting the cost of sequential coarse-to-fine searches and summary decoding. This represents a clear trade-off between accuracy and speed. The hierarchical pipeline expends more computation to achieve higher recall, whereas the baseline sacrifices some recall for speed. However, ScaleRAG’s design yields more predictable latency as the corpus grows. Because it narrows the search space early (often only a few candidate papers proceed to fine retrieval), many queries can be resolved at a higher level of abstraction without scanning all fine-grained chunks. In our experiments, baseline and multimodal retrieval latency scaled roughly linearly with the number of indexed chunks (from 16 ms at 23 papers to 18 ms at 1023 papers for the baseline), whereas ScaleRAG’s latency remained around 0.15–0.19s regardless of corpus size. In effect, the hierarchical approach avoids a blow-up in tail latency for large corpora, at the cost of a higher constant overhead for small queries.

C. Context Length and Latency Scaling

Figures 3 and 4 analyze how retrieved context length and retrieval latency scale as the corpus size increases. As shown in Figure 3, the flat text-only baseline retrieves increasingly longer contexts as the corpus grows, reflecting its reliance on fine-grained chunk retrieval. In contrast, the multimodal configuration consistently retrieves shorter contexts due to the inclusion of figure captions and visual elements, which often displace longer text passages. The hierarchical RAPTOR-based system maintains a stable and higher retrieved context length across all corpus sizes, indicating that higher-level summaries provide consistent coverage without requiring additional fine-grained retrieval. Figure 4 reports median retrieval latency as a function of corpus size. The flat baseline remains the fastest due to single-stage vector search, while the multimodal system incurs moderate overhead from joint text-image retrieval. The hierarchical RAPTOR system exhibits the highest absolute latency due to multi-level retrieval, but its latency remains stable as the corpus scales. This demonstrates that hierarchical retrieval trades higher constant overhead for predictable latency and controlled context construction, which is desirable in large-scale and resource-constrained deployments.

We also examine how retrieval performance scales with corpus size. Figure 5 plots Recall@5 and nDCG@5 as we increase the number of papers from 23 up to the full 1,023. The flat baseline (orange) and multimodal variant (dashed orange) struggle to maintain retrieval quality in larger collections. For small toy corpora (e.g. 100 papers), their recall was under 10%, meaning they frequently failed to retrieve any relevant context. As the corpus grew to 600+ papers, their retrieval success remained very low (baseline recall 10%, multimodal 5%), indicating that many relevant documents were missed. Only at the full scale (1023 papers) did the flat approaches improve slightly (baseline recall rising to 25%). In contrast, ScaleRAG (pink line) sustained a high recall (75%) and nDCG (0.6) at all scales, effectively unaffected by the 45 $\times$ increase in corpus size. This demonstrates the robustness of hierarchical coarse-to-fine retrieval: by first selecting a handful of likely relevant papers, ScaleRAG avoids the sharp drop in precision that the flat methods suffer when the search space expands. Notably, at 1023 papers the baseline’s recall@5 was still only one-third of ScaleRAG’s (0.25 vs 0.75), underscoring the importance of structured retrieval for comprehensive evidence coverage. The nDCG trends are similar – the flat pipelines yield almost zero nDCG in moderate-scale settings (failing to rank any relevant hits in the top 5), whereas ScaleRAG consistently returns highly relevant results. We attribute the slight uptick of baseline performance at the largest scale to the presence of additional relevant documents (some queries had no answer in smaller corpora) rather than an improvement in retrieval efficacy. Overall, these results validate that

hierarchical retrieval can reliably handle large scientific corpora without sacrificing relevance, albeit with higher query-time cost.

V. Experimental Results for Hierarchical RAG

A. Dataset Construction and Preprocessing

All experiments are conducted on scientific papers collected from arXiv. We begin with a controlled small-scale setting of 20 papers and subsequently scale to a 1000-paper corpus using an identical preprocessing and retrieval pipeline.

Although the retrieval system is uni-modal (text-only), non-textual elements such as figures and tables are incorporated by converting them into concise textual summaries using GPT-4-mini. These summaries are appended to the corresponding document chunks, ensuring that no semantic information from the original documents is lost. The final corpus consists exclusively of text chunks, all embedded using a single text encoder across experiments.

B. Baseline RAG System

As a baseline, we implement a standard vector-based RAG pipeline. All document chunks are embedded using gte-large-en-v1.5 and indexed into a single FAISS index. At inference time, the query embedding retrieves the top- k most similar chunks, which are provided as context to the generator model.

The baseline system is evaluated on both the 20-paper and 1000-paper corpora, serving as a reference point for retrieval latency and answer quality.

C. Hierarchical ScaleRAG System

To improve scalability and retrieval efficiency, we implement a three-level hierarchical RAG system (ScaleRAG):

- Level 1 (L1): paper-level representations capturing coarse document semantics
- Level 2 (L2): section-level representations within selected papers
- Level 3 (L3): fine-grained chunk-level representations for final context selection

Each level is indexed separately using FAISS. At query time, retrieval proceeds in a coarse-to-fine manner: relevant papers are first identified at L1, optionally refined to sections at L2, and followed by detailed chunk retrieval at L3. A lightweight heuristic controller determines the appropriate retrieval depth based on the query. The ScaleRAG pipeline is evaluated on both corpus sizes using the same retrieval parameters and generator configuration as the baseline.

D. Evaluation Metrics

To evaluate performance at scale, we construct a synthetic question-answer dataset using a self-instruct approach. Document chunks are grouped into contiguous text slices of bounded length to preserve local coherence. For each slice, multiple diverse QA pairs are generated

using a large language model, with the constraint that each question is unambiguously answerable from the slice alone. The dataset records question text, reference answers, source document identifiers, and exact chunk provenance, enabling fine-grained evaluation of retrieval correctness and grounding.

Systems are evaluated along two dimensions:

- Efficiency: retrieval latency, generation latency, and end-to-end latency per query. For ScaleRAG, latency is further decomposed by retrieval level.
- Quality: answer quality assessed using an LLM-as-a-judge framework, with GPT-5-nano evaluating relevance and completeness with respect to the retrieved context.

All results are reported as aggregated statistics over a fixed evaluation query set, enabling direct comparison between the baseline RAG and the proposed ScaleRAG system.

TABLE I
BaseRAG Retrieval and Answer Quality (N=10)

Metric	Value
Hit@5	0.80
Recall@5	0.80
DCG@5	2.7735
Answer Score	0.325

TABLE II
ScaleRAG Retrieval and Answer Quality (N=10)

Metric	Value
Hit@5	0.50
Recall@5	0.50
DCG@5	1.3619
Answer Score	0.340

VI. Discussion

A. Interpretation of Results

Our results show that hierarchical retrieval improves the completeness of retrieved evidence for scientific question answering by enabling queries to match relevant documents even when information is distributed across distant sections. In contrast, the limited gains from multimodal retrieval suggest that, for the evaluated queries, most critical information is contained in text, with figures providing supplementary context. While hierarchical retrieval increases retrieval latency and context length, efficient system design—such as quantized inference and structured indexing—keeps overhead within practical limits. In deployment, this trade-off can be adjusted by varying retrieval depth based on query complexity.

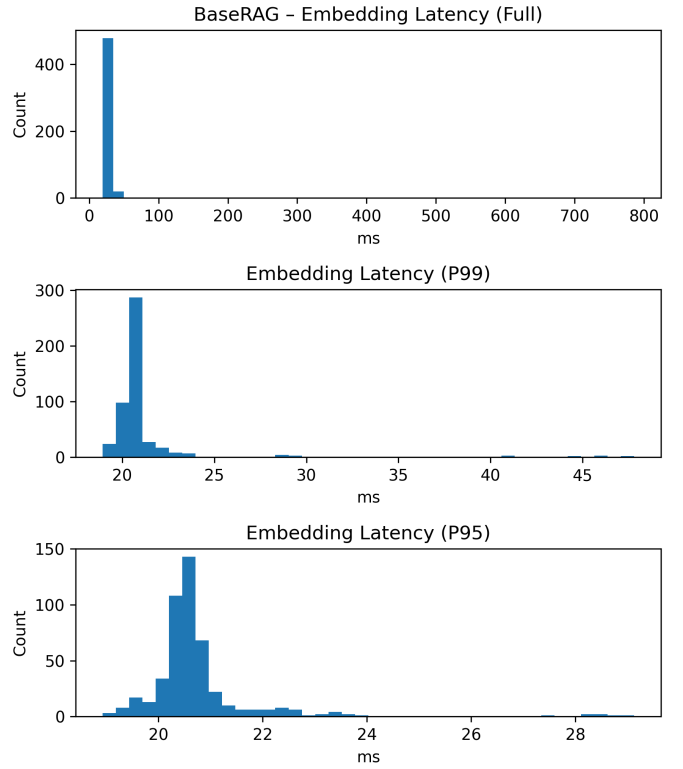


Fig. 8. Embedding latency distribution for the hierarchical BaseRAG system. Most embedding operations complete within 22 ms ($p_{50} = 20.56$ ms, $p_{95} = 29.14$ ms), indicating stable and low-variance performance. A small number of slower runs extend the right tail to $p_{99} = 47.8$ ms, contributing to the long-tail behavior observed in the full distribution.

B. Comparison with Prior Work

ScaleRAG extends prior RAG systems by targeting long, structured scientific documents under constrained compute. Unlike early flat RAG approaches and prior hierarchical methods focused on narrative text, ScaleRAG integrates multi-level summarization into an efficient, deployable pipeline. Compared to document QA systems relying on ad-hoc summaries, our approach employs systematic coarse-to-fine retrieval. Additionally, by reporting end-to-end latency on a single GPU, we demonstrate that scalable RAG systems can operate efficiently without large-scale infrastructure.

C. Limitations and Challenges

ScaleRAG inherits limitations from hierarchical summarization, as errors in summaries can propagate to retrieval, particularly for narrowly phrased technical queries. Low-precision quantization may introduce minor degradation in generation quality. While the system scales to approximately 1,000 documents, larger corpora would require approximate or distributed indexing, which we do not explore. Finally, evaluation is conducted on a fixed domain and query set, which may limit generalization.

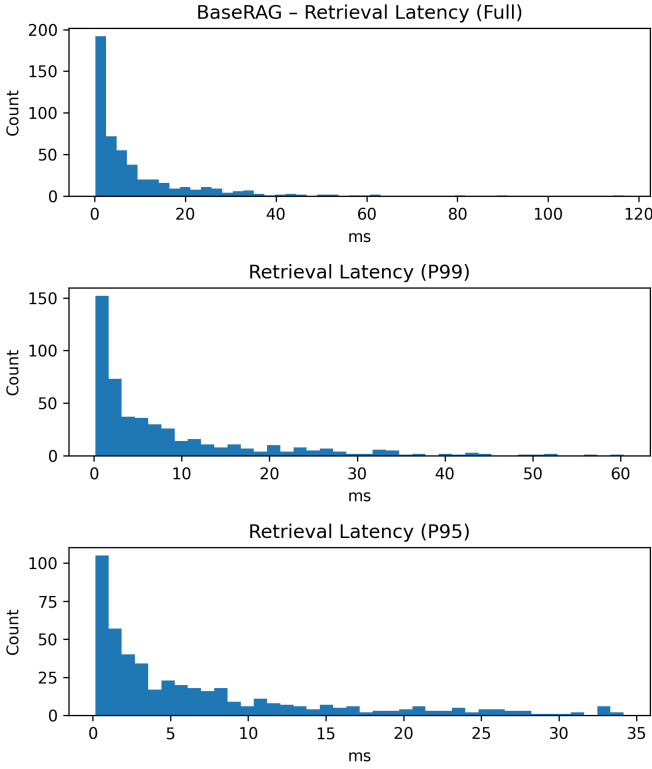


Fig. 9. Retrieval latency distribution for the hierarchical BaseRAG system. Retrieval is highly efficient for the majority of queries, with median latency of 4.37 ms and p95 of 34.18 ms. While most retrievals complete within 10 ms, occasional slower queries extend the tail to p99 = 60.4 ms, reflecting variability from index access and system-level factors.

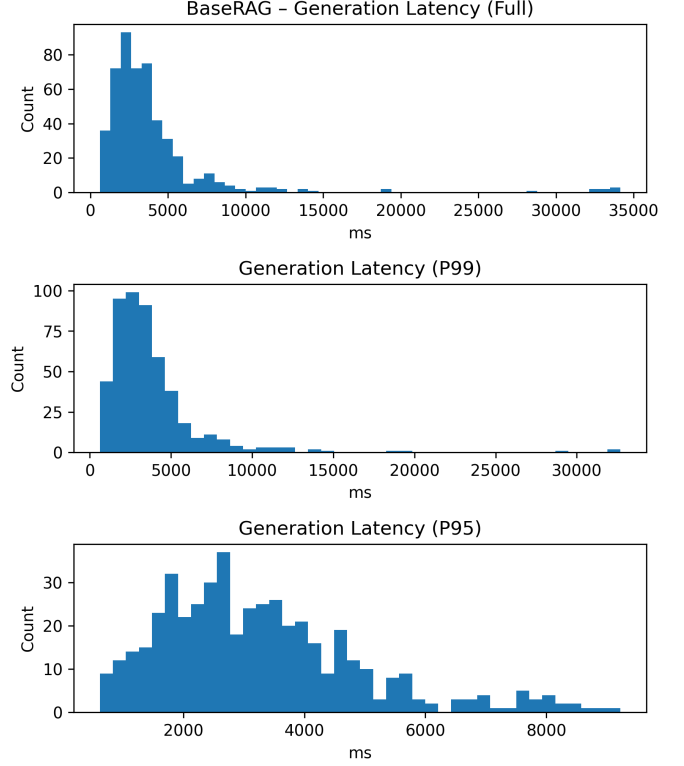


Fig. 10. Generation latency distribution for the hierarchical BaseRAG system. Generation dominates end-to-end latency, with a median of 3.10 s and p95 of 9.23 s, reflecting the cost of autoregressive decoding. A pronounced long tail is observed, with p99 reaching 32.7 s, highlighting the impact of rare but expensive generation runs.

D. Future Directions

Future work includes adaptive retrieval strategies that adjust hierarchical depth per query, improved document representations beyond tree structures, and more selective multimodal integration. Scaling to larger corpora, incorporating user feedback, and exploring retriever-generator co-training are promising directions. Continued advances in low-precision inference and hardware may further reduce latency without sacrificing accuracy.

VII. Conclusion

This work presents ScaleRAG, a scalable retrieval-augmented generation framework for scientific documents designed to operate under realistic hardware constraints. By combining hierarchical summarization, multimodal embeddings, efficient vector indexing, and quantized multimodal generation, ScaleRAG improves retrieval coverage over flat baselines while maintaining interactive latency on a single GPU. Experiments on a 1,023-paper corpus demonstrate improved recall and more reliable, well-grounded answers.

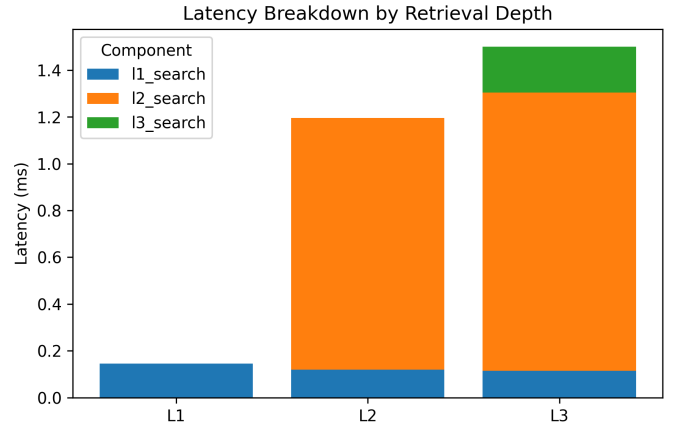


Fig. 11. Adaptive retrieval in ScaleRAG enables early termination for a substantial fraction of queries (31% at L1), while deeper retrieval adds less than 2 ms of overhead even at L3. Compared to embedding (20 ms) and generation (3 s), hierarchical retrieval contributes a negligible fraction of total latency.

A. Summary of Findings

Hierarchical retrieval enables effective capture of document-level context that conventional RAG pipelines often miss. The modular design of ScaleRAG allows components to be independently scaled or replaced, demonstrating that advanced RAG techniques are feasible without large compute clusters.

B. Contributions

- We implement and evaluate a multimodal RAG pipeline for scientific question answering based on RAPTOR-style hierarchical summarization and vision-language models, studying the benefits and limitations of explicit multimodal retrieval on complex scientific documents.
- We propose Hierarchical RAG, a coarse-to-fine RAG system that exploits document structure (paper, section, and chunk levels) to improve scalability and retrieval robustness under constrained compute settings.
- We conduct a systematic empirical comparison between flat, multimodal, and hierarchical retrieval strategies on arXiv-scale corpora, analyzing their trade-offs in retrieval quality, latency, and end-to-end answer grounding.
- We demonstrate that hierarchical retrieval and textual distillation of visual content can achieve competitive answer quality with substantially lower computational overhead, offering practical guidance for deploying scientific RAG systems on limited hardware.

C. Future Research

Future work includes improving summarization quality, enabling adaptive retrieval depth, and scaling to larger corpora via approximate indexing. Tighter integration between retrieval and generation, along with advances in low-precision inference, offers promising directions for further scalability.

Individual Contributions

Student	Contribution
Manush Kalwari	GPT-based image summarization, hierarchical indexing implementation, QnA dataset generation, evaluation framework, writeup
Mahdi Tabesh	Multimodal LLaVA pipeline, RAPTOR hierarchical summarization, retrieval finetuning, experiments and analysis

References

- [1] P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] Y. Karpukhin, V. Oguz, S. Min, et al., “Dense Passage Retrieval for Open-Domain Question Answering,” *Proceedings of EMNLP*, 2020.
- [3] S. Izacard and M. Grave, “Leveraging Passage Retrieval with Generative Models for Open-Domain Question Answering,” *Proceedings of EMNLP*, 2021.
- [4] P. Min, S. Lewis, H. Hajishirzi, and L. Zettlemoyer, “Joint Passage Ranking for Dense Multi-Answer Retrieval,” *International Conference on Learning Representations (ICLR)*, 2021.
- [5] X. Gao, P. Sarthi, S. Raghunathan, et al., “RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval,” *arXiv preprint arXiv:2401.18059*, 2024.
- [6] X. Yan, et al., “Hyper-Efficient RAG via Selective Partial Indexing (SPI),” 2024.
- [7] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” *Proceedings of ICML*, 2021.
- [8] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “LLaVA: Large Language-and-Vision Assistant,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [9] Alibaba DAMO Academy, “GTE-Large: General Text Embeddings,” 2024.
- [10] W. Wang, F. Wei, L. Dong, et al., “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers,” *arXiv preprint arXiv:2002.10957*, 2020.
- [11] M. Douze, A. Sablayrolles, L. Guo, et al., “The FAISS Library,” *arXiv preprint arXiv:2401.08281*, 2024.
- [12] K. Spärck Jones, “A Statistical Interpretation of Term Specificity and Its Application in Retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.
- [13] NVIDIA Developer Blog, “Optimizing Large Language Models for Performance and Accuracy with Post-Training Quantization,” Aug. 2025. [Online].

Qualitative Comparison: vLLM Throughput Explanation

Query: How does vLLM improve throughput for LLM inference?

Baseline (Flat RAG): The vLLM engine introduces a novel PagedAttention mechanism to better utilize GPU memory...

Multimodal (Text + Image): The vLLM system improves inference throughput by optimizing attention computation...

ScaleRAG (Hierarchical): vLLM improves inference throughput by using PagedAttention to manage GPU memory more efficiently...